GatePass

Comprehensive Q&A — Visitor Gate Pass & On-Site Register

Client: NubTech Industries Pvt. Ltd.

Stack: SAP CAP · Node.js · OData V4 · Fiori Elements · HANA Cloud

Scope: 1 Document · 14 Business Rules · 8 Statuses · 4 Roles

Prepared by: AI Assistant — DataNub Enablement

Date: 18 September 2026

GatePass — Comprehensive Q&A

Every answer below has two parts: LAYMAN **Layman** (plain English, no jargon) and TECHNICAL **Technical** (CAP/SAP developer terms).

PART 1 — BUSINESS CONTEXT

1.1 Who is the client, and why BTP instead of a standard SAP transaction?

LAYMAN **Layman:**

The client is **NubTech Industries Pvt. Ltd.** — a manufacturing company with 1,400 employees across four plants (Chennai, Hosur, Pune, Baddi). They need a system to track who enters and leaves their factory gates. No SAP standard transaction exists for visitor gate management — SAP covers purchase orders, inventory, payroll, etc., but nobody at SAP built a "visitor register" module. So this becomes a **custom side-application** built on SAP's cloud platform (BTP), running alongside the ERP but independent of it.

TECHNICAL **Technical:**

SAP BTP (Business Technology Platform) is the correct home for a **white-space process** — a business need that falls outside the standard SAP S/4HANA transaction catalogue. There is no `T-Code` for gate pass management. Building it on BTP with the **Cloud Application Programming Model (CAP)** gives you OData V4 services, Fiori Elements UI generation, HANA Cloud persistence, and XSUAA-based authorization — all without touching the ERP core. The app is a pure **side-by-side extension**.

1.2 What is wrong with the paper gate register today? Name four problems.

#	Problem	Business Impact
P1	Paper register, handwritten by the visitor at the gate	Names and mobile numbers are illegible, unsearchable, unreportable
P2	No pre-registration — the guard telephones the host on arrival	Visitor waits ~11 minutes at the gate while the host is hunted down
P3	The exit column is filled in only when somebody remembers	The register shows people "inside" who left days ago — the evacuation muster list is fiction
P4	Safety induction validity is never checked at the gate	Contractors reach the shop floor on an expired induction — statutory violation
P5	Badges come off a hook board with no record	Badges are not returned, cannot be traced to a person
P6	Laptops, tools and samples walk in and out unrecorded	No proof of what came in, disputes are unresolvable

1.3 Which problem matters most, and why?

LAYMAN **Layman:**

P3 — the exit column nobody fills in. The other problems are annoying or inefficient. P3 is dangerous. If there is a fire and the security officer cannot instantly say who is inside the building, people die. A fire drill in March 2026 took 34 minutes to produce a muster list against an 8-minute target. A factory inspector can shut you down for that. Every "fussy" design decision in this app — check-out being its own action, never auto-closing a visitor, flagging overstay instead of silently marking them as "left" — exists because of P3.

TECHNICAL **Technical:**

P3 drives the architectural decisions: `CHECK_OUT` is a mandatory explicit action (not a background process), `ON_SITE → CANCELLED` is a forbidden transition, and the day-end job flags overstays (`isOverstay = true`) without inventing a departure time. The system never fabricates an `actualOutAt` timestamp. The "On Site Now" report (R1) derives its data from `status_code = 'ON_SITE'`, making it correct by construction rather than by somebody remembering to fill a column.

1.4 Name the four user roles and what each one does.

Persona	Role Code	What They Do	Volume
Host (any employee)	`Host`	Pre-registers a visitor, declares items, submits, tracks, cancels. Sees **only their own** passes.	~1,400 users
Security Supervisor	`SecuritySupervisor`	Approves or rejects passes for approval-relevant areas **at their plant**. Reads all passes at that plant.	4 users
Security Guard	`SecurityGuard`	Works the gate console: verifies ID, issues badge, checks in, verifies items, checks out. **At their plant only.**	12 users, 3 shifts
Administrator	`Admin`	Maintains master/config data, runs the day-end job. Reads everything. **Never approves, never operates the gate.**	2 users

LAYMAN **Layman:** Think of it as separation of duties. The person who requests the visitor can never approve them. The person who approves can never operate the gate. The guard admits but never approves. This is what an auditor checks first.

TECHNICAL **Technical:** Implemented via `@restrict` annotations with `where` clauses in gatepass-service.cds. Instance-based authorization (`where: 'createdBy = $user'`) restricts a Host to their own passes. Plant-level restriction (`where: 'plant.code = $user.plant'`) scopes Guards and Supervisors.

1.5 Name three things that are deliberately out of scope.

1. **All hardware** — turnstiles, boom barriers, biometrics, RFID, cameras, photo capture, badge printers. The guard types a badge number.
2. **SMS or e-mail notification** — the worklists (approval list, gate console) are the notification mechanism.
3. **Vehicle and material gate entry as documents of their own** — items are declared on the visitor pass, not tracked as separate documents.

Also out of scope: the safety induction itself (the app only stores the reference and validates the date), multi-level approval chains, ERP integration, and native mobile apps.

LAYMAN **Layman:** Writing this list *before* anyone asks is a consultant habit. Most project failures happen because two people assumed different things. One of them is picturing turnstiles.

PART 2 — ARCHITECTURE & CAP

2.1 What are the main folders in a CAP project, and what goes in each?

Folder	What Goes In	In This Project
`db/`	The **data model** — CDS entity definitions and the CSV seed data	schema.cds defines all entities; `db/data/` holds CSV files
`srv/`	The **service layer** — CDS service definitions and JavaScript/TypeScript handlers	gatepass-service.cds (projections, actions, auth) and gatepass-service.js (business logic)
`app/`	The **UI** — Fiori Elements annotations, i18n, manifest files	Sub-folders per app: `my-passes/`, `approvals/`, `gate-console/`, `reports/`, admin apps, plus common.cds for shared annotations
`test/`	**Automated tests**	Test scripts using `@cap-js/cds-test`

LAYMAN **Layman:** Think of it as three layers of a building. The basement (`db/`) stores data. The ground floor (`srv/`) runs the business rules. The top floor (`app/`) is what people see on screen.

TECHNICAL **Technical:** CAP's convention-over-configuration means `cds watch` auto-discovers `db/`, `srv/`, and `app/`. The `db/data/` CSV convention (`namespace-EntityName.csv`) auto-seeds on `cds deploy`. The `gen/` folder holds compiled EDMX and SQL output.

2.2 Why is the database model not exposed directly to the UI? What sits in between?

LAYMAN **Layman:**

Imagine a bank vault. You don't let customers walk into the vault and grab cash. Instead, there's a counter (the **service layer**) in between. The teller checks your ID, applies rules, and hands you the money. The service layer is that teller — it decides what data to show, what to hide, and what rules to enforce.

TECHNICAL **Technical:**

The **service definition** (gatepass-service.cds) creates **projections** (`entity VisitorPasses as projection on gatepass.transaction.VisitorPasses`) that sit between the database model and the UI. This layer:

1. **Controls the shape** — can add virtual fields (e.g., `ageingDays`), exclude internal fields, rename things
2. **Enforces authorization** — `@restrict` annotations with `where` clauses
3. **Defines actions** — `submit()`, `approve()`, `checkIn()`, etc.

4. **Adds capabilities** — `@odata.draft.enabled`, `@readonly`, search annotations

The UI talks to the OData V4 service, never to the database tables directly.

2.3 Should business rules be in the Fiori UI or in the server handlers? Why?

LAYMAN **Layman:**

Always on the server. If you put the rule in the UI, it only works as long as someone uses that specific screen. Anybody with a URL and a tool like `curl` can bypass it completely. The spec says (TC-08): "Send a raw PATCH setting actualInAt, passNo and status — all refused. This is the test that proves the rules live on the server."

TECHNICAL **Technical:**

Business rules are implemented in gatepass-service.js as **before/on/after handlers**. The UI can provide convenience (hiding a button, showing a warning), but the *enforcement* must be server-side. CAP handlers run for every caller — Fiori Elements, REST clients, test scripts, OData batch calls. The spec explicitly says: "Enforce it on the server, not by leaving the button off the UI."

For example, `idProofNumber` masking (BR-14) is done in an `after('READ')` handler, not by a UI formatter. Hiding the "Approve" button from the host is nice UX, but the server must still return `403` if they call the action directly.

2.4 Why is "requires approval" a configuration flag rather than an IF statement?

LAYMAN **Layman:**

Imagine the security team says: "Starting next month, the canteen also needs approval." If that rule is coded as `if (area === 'SHOPFLOOR' || area === 'WAREHOUSE')`, a developer has to change the code, test it, and deploy it. That takes days and introduces risk. Instead, the VisitAreas table has a column called `requiresApproval`. The admin ticks a checkbox in the master data screen. Done. No developer needed.

TECHNICAL **Technical:**

BR-04 is a **determination, not validation**. The submit handler reads `area.requiresApproval` and routes to `REQUESTED` or `APPROVED`. This is the "configuration-versus-code lesson" the spec calls out. The same applies to `requiresInduction` (BR-05). Both are Boolean columns on the `VisitAreas` entity. The code in the handler is:

```
const target = area?.requiresApproval ? 'REQUESTED' : 'APPROVED';
```

One line of generic code, infinite configuration combinations. TC-01 proves it: OFFICE (no approval) → straight to APPROVED. SHOPFLOOR (approval required) → REQUESTED.

2.5 EHS decides that from 1 October the R&D lab also needs a safety induction. What has to change?

LAYMAN **Layman:**

An administrator opens the Visit Areas master data screen, finds the "R&D Laboratory" row for the Chennai plant, and ticks the "Requires Induction" checkbox. Save. Done. From that moment, anyone submitting a pass to the R&D lab must provide an induction reference number and a valid expiry date, or the system refuses. **No code changes. No deployment. No developer.**

TECHNICAL **Technical:**

Update the `RNDLAB` row in `VisitAreas` where `plant_code = '1000'`: set `requiresInduction = true`. The submit handler already checks `area.requiresInduction` (BR-05) generically. The check-in handler re-validates it. The test data in Section 12.2 shows RNDLAB currently has `requiresInduction: false` — flipping it is a data operation, not a code change.

> *The spec says: "If any part of your answer is 'change the code', redesign the model — that request should never reach a developer at all."*

2.6 Why is passNo assigned at submit rather than when the draft is created?

LAYMAN **Layman:**

Think of it like a cheque book. You don't number the cheques before deciding whether to write them. A draft is just a rough note — the host might never finish it, or might delete it. If you hand out pass numbers at draft creation, you'd burn through numbers for passes that never exist. Also, you'd have "gaps" in the number series that security can't explain to an auditor.

TECHNICAL **Technical:**

The pass number format is `GP---<5 digits>`, generated sequentially per plant per year (BR-12). Assigning at draft time would:

1. **Waste numbers** — drafts that are abandoned or deleted would leave gaps
2. **Create concurrency problems** — multiple users creating drafts simultaneously would need pessimistic locking just to hold a number
3. **Violate the contract** — a draft has no rules applied yet; it's private and invisible to everyone else. Giving it an official number implies it's a real pass.

In the code (gatepass-service.js:431):

```
const passNo = await generatePassNo(pass.plant_code);
```

This runs only in the `submit` action handler, after all validations pass.

2.7 Why is the status history kept in a separate PassEvents table instead of 'changedBy / changedOn' fields on the pass?

LAYMAN **Layman:**

If you only keep "last changed by" on the pass, you know who did the *most recent* thing — but not who submitted it, who approved it, or when the guard checked the visitor in. It's like a notebook where every entry overwrites the previous one. You lose the story. PassEvents is the full diary: every action, who did it, when, and what the status was before and after.

TECHNICAL **Technical:**

`PassEvents` is an **immutable append-only audit log** — one row per status transition. Each row records `fromStatus`, `toStatus`, `action`, `actor` (from `req.user.id`), `actedAt` (server timestamp), and an optional `remark`. The `managed` aspect on `VisitorPasses` gives you `createdBy/createdAt/modifiedBy/modifiedAt` — that's the "who touched it last" snapshot. PassEvents gives you the complete lifecycle trace: DRAFT → REQUESTED → APPROVED → ON_SITE → CHECKED_OUT, with every actor and timestamp.

This is critical for:

- **Audit compliance** — an auditor can reconstruct who approved what and when
- **Dispute resolution** — if a badge goes missing, you can prove the exact timeline
- **Immutability** — even an Admin cannot modify or delete event rows (TC-10 tests this)

2.8 Why one transition table used by every action, instead of checking inside each handler?

LAYMAN **Layman:**

Imagine seven different guards each carrying their own list of which doors they can open. If one list has a typo, that door is now insecure. Instead, you have *one master keycard list* at the front desk. Everyone checks the same list. One place to update, one place to audit, zero discrepancies.

TECHNICAL **Technical:**

The transition table is `ACTION_MAP` at the top of gatepass-service.js:

```
const ACTION_MAP = {
  submit:     { from: ['DRAFT'],                          to: null },
  approve:    { from: ['REQUESTED'],                      to: 'APPROVED' },
  rejectPass: { from: ['REQUESTED'],                     to: 'REJECTED' },
  cancel:     { from: ['DRAFT', 'REQUESTED', 'APPROVED'], to: 'CANCELLED' },
  checkIn:    { from: ['APPROVED'],                       to: 'ON_SITE' },
  checkOut:   { from: ['ON_SITE'],                        to: 'CHECKED_OUT' },
  expire:     { from: ['APPROVED'],                       to: 'EXPIRED' },
  overstay:   { from: ['ON_SITE'],                      to: 'ON_SITE' }
};
```

Every action calls `assertTransition(currentStatus, actionName, req)`. Benefits:

1. **Single source of truth** — one table to read, audit, and test
2. **Forbidden transitions are explicit by absence** — ON_SITE → CANCELLED is simply not in the table
3. **Consistent error handling** — the same `409 INVALID_STATUS` response for any illegal move
4. **Easier to extend** — adding a new transition is one line, not scattered across seven handlers

---

2.9 How does the same application run on SQLite locally and on SAP HANA Cloud on BTP?

LAYMAN **Layman:**

CAP acts as a translator. You describe your data in a language called CDS (like a blueprint). When you run locally, CAP reads the blueprint and creates SQLite tables. When you deploy to the cloud, it reads the same blueprint and creates HANA tables. Your code doesn't know or care which database is underneath — it speaks to CAP, and CAP speaks to whichever database is present.

TECHNICAL **Technical:**

CDS (Core Data Services) is **database-agnostic**. The same schema.cds compiles to different SQL dialects:

- **Local development:** `@cap-js/sqlite` is in `devDependencies` in package.json. `cds watch` → SQLite in-memory or file-based.
- **Production:** `@cap-js/hana` is in `dependencies`. The `[production]` profile in `package.json` sets `auth.kind: 'xsuaa'`. The `mta.yaml` deploys the HDI container to HANA Cloud.

The CQL (CDS Query Language) API — `SELECT.from()`, `INSERT.into()`, `UPDATE()` — is compiled by the CAP runtime to the appropriate SQL dialect. As long as you use CQL and not raw SQL, portability is automatic. The spec says: "Same model file runs on HANA" is a Day 5 gate criterion.

PART 3 — DATA MODEL

3.1 List the entities — documents, children, master, and configuration data.

Entity	Kind	Description
VisitorPasses	Transactional document (header)	One planned or actual visit by one visitor. The **only document** in this app.
PassItems	Owned child (Composition)	One line per item carried in (laptop, toolkit, sample). Returnable flag, returned-at-exit flag.
PassEvents	Owned child (Composition, write-once)	One row per status change. Never updated or deleted by anybody.
Employees	Master data	Every employee who can be a host. Has default plant, active flag.
Plants	Master data	The four sites, with gate opening/closing hours.
VisitAreas	Configuration data	Office, shop floor, warehouse, lab. Carries `requiresApproval` and `requiresInduction` — the two flags that drive the entire process.
VisitPurposes	Code list	SUPPLIER, CUSTOMER, INTERVIEW, SERVICE, COURIER, GOVT, OTHER
IdProofTypes	Code list	AADHAAR, PAN, DL, PASSPORT, EMPCARD
PassStatus	Code list	DRAFT, REQUESTED, APPROVED, REJECTED, ON_SITE, CHECKED_OUT, CANCELLED, EXPIRED — with criticality for Fiori badge colours

---

3.2 Composition vs. Association — give one example of each.

LAYMAN **Layman:**

- **Composition** = "part of." A pass item (a laptop listed on a pass) doesn't make sense without the pass. Delete the pass → the item is deleted too. It's like a page in a book — rip out the book, the page goes with it.
- **Association** = "refers to." A pass *refers to* an employee as the host. The employee exists independently. Deleting the pass must never delete the employee. It's like writing someone's phone number on a sticky note — throwing away the note doesn't erase the person.

TECHNICAL **Technical:**

Composition example — Pass → PassItems (schema.cds:132-133):

```
items : Composition of many PassItems on items.pass = $self;
```

This means deep insert, deep update, and cascade delete. A `POST` to the pass with inline items creates everything in one request.

Association example — Pass → Employees (schema.cds:103):

```
host : Association to master.Employees;
```

This creates a foreign key (`host_ID`) but no ownership. Deleting a pass never touches the Employees table.

3.3 Which three fields must the server always set, and never accept from the browser?

LAYMAN **Layman:**

1. **`passNo`** — the official pass number. If a user could type it, they could create duplicates or forge numbers.
2. **`actualInAt`** — the check-in timestamp. If the browser could set this, someone could fake a check-in.
3. **`actualOutAt`** — the check-out timestamp. If the browser could set this, someone could give a visitor a departure time they never had — and the whole register becomes worthless (back to P3).

TECHNICAL **Technical:**

These three (plus `status_code` , `badgeNo` , `durationMinutes` , `isOverstay` , `rejectionReason` , `host_ID`) are listed in the `SYSTEM_FIELDS` array (gatepass-service.js:31-35):

```
const SYSTEM_FIELDS = [
  'passNo', 'actualInAt', 'actualOutAt', 'durationMinutes',
  'isOverstay', 'badgeNo', 'status_code', 'status',
  'rejectionReason', 'host_ID', 'host'
];
```

The `stripSystemFields` handler runs `before('UPDATE')` on both the draft entity and the active entity. It `delete` s these keys from `req.data` silently. TC-08 sends a raw PATCH with all three — nothing changes in the database.

3.4 Why is the host's name never stored on the pass itself?

LAYMAN **Layman:**

If you copy someone's name onto the pass, you now have two copies of the name — one on the pass, one on the employee record. When Anitha Raman gets married and changes her last name, every old pass still shows the old name. Or worse, you update some passes but forget others. One source of truth is always better than two.

TECHNICAL **Technical:**

The pass stores `host_ID` (a foreign key / Association to `Employees`), not `host_name`. The name is **read through the association** at query time: `$expand=host` resolves to the current employee record. This is the **no-copy-of-master-data** principle. The spec says: "never copy the name onto the pass, read it through the association." The rubric docks marks for "copies of data that belongs to master data."

3.5 What is a code list in CAP, and why use one instead of a plain text field?

LAYMAN **Layman:**

A code list is a fixed menu of valid choices — like a dropdown. Instead of letting the user type "Supplier", "supplier", "SUPPLIER", "Vendor meeting", you give them a list: SUPPLIER, CUSTOMER, INTERVIEW, etc. Each has a code (what the computer stores) and a description (what the user sees). This prevents typos, enables translation, and makes reporting reliable.

TECHNICAL **Technical:**

CAP's `sap.common.CodeList` aspect provides `name` and `descr` fields with i18n support out of the box. The entities extend it:

```
entity VisitPurposes : CodeList { key code : String(12); }
```

Benefits:

1. **Referential integrity** — only valid codes are accepted
2. **Translation** — `name` and `descr` support locale-specific text via `_texts` tables
3. **Value helps** — Fiori Elements automatically renders dropdowns from code lists
4. **Reporting** — aggregation by `purpose_code` gives consistent, comparable results

---

3.6 What does the `managed` aspect give you, and why should you never write those fields yourself?

LAYMAN **Layman:**

The `managed` aspect is like a security camera on a filing cabinet. Every time someone opens the cabinet (creates or updates a record), the camera automatically stamps: "Created by Anitha on 18-Sep-2026 at 3pm" or "Last changed by Karthik on 19-Sep-2026 at 10am." You don't write this yourself — the camera does it. If you wrote it yourself, you could lie.

TECHNICAL **Technical:**

The `managed` aspect (from `@sap/cds/common`) adds four fields: `createdAt`, `createdBy`, `modifiedAt`, `modifiedBy`. CAP's generic handlers populate them automatically from the request context (`req.user.id` and `req.timestamp`). Writing them manually would:

1. **Allow spoofing** — a user could claim another user created the record
2. **Be redundant** — CAP already does it

3. **Break instance-based auth** — the `where: 'createdBy = $user'` clause in `@restrict` depends on CAP setting `createdBy` correctly

The spec says: "Never write these yourself. CAP maintains them."

3.7 PassEvents is described as 'write once'. What does that mean, and how do you enforce it?

LAYMAN **Layman:**

"Write once" means: once a record is written, nobody — not the admin, not the person who wrote it, not even the system itself — can change or delete it. Ever. It's like writing in permanent marker. This is the audit trail. If an event could be edited, someone could erase the fact that a visitor was checked in, and you're back to the paper register problem.

TECHNICAL **Technical:**

Enforced at three levels:

1. **CDS annotation** — `@readonly` on the service projection (gatepass-service.cds:178) blocks CREATE, UPDATE, DELETE at the OData protocol layer
2. **Handler belt** — `before(['CREATE', 'UPDATE', 'DELETE'], PassEvents)` rejects with `403 EVENTS_IMMUTABLE` (gatepass-service.js:313-314)
3. **Authorization** — no role is granted `UPDATE` or `DELETE` on `PassEvents`

The only way events are written is through the internal `writeEvent()` helper function, which uses the `db()` entities (bypassing service-level restrictions) inside action handlers. TC-10 tests this: "Update, then delete, a PassEvent row over OData — both refused."

3.8 How is a master record taken out of use, and why is it never deleted?

LAYMAN **Layman:**

You flip the "Active" switch to off. The record is still there — old passes can still show "this visit was at Plant 4000" — but it disappears from the dropdown menus for new passes. You never delete it because old passes reference it. If you deleted Plant 4000, every pass that was issued for that plant would have a broken link — it would say "plant: ???" instead of a name.

TECHNICAL **Technical:**

Every master entity (`Employees` , `Plants` , `VisitAreas`) has an `isActive : Boolean default true` field. The service grants no role the `DELETE` operation — both `@Capabilities.DeleteRestrictions.Deletable: false` (hides the UI button) and a handler-level `before('DELETE')` reject (prevents raw HTTP DELETE). Value helps filter to `isActive = true` for new data entry, but old passes resolve the association regardless of the active flag. This is the **soft-delete** pattern.

3.9 Which fields are mandatory only in certain situations? Name them and the condition.

Field	Condition
`purposeDetail`	Mandatory when `purpose_code = 'OTHER'`, minimum 15 characters
`inductionRef`	Mandatory when the selected area has `requiresInduction = true`
`inductionValidTill`	Mandatory alongside `inductionRef`; must be ≥ `visitDate`
`rejectionReason`	Mandatory on the `rejectPass` action, minimum 10 characters (BR-10)
`exitRemark`	Mandatory on a returnable item when the guard does NOT confirm its return (BR-09)
`badgeNo`	Mandatory at check-in (BR-08), but not at any other stage

TECHNICAL **Technical:** These are **conditionally mandatory** (`Cond.` in the spec). They cannot be enforced with a simple `@mandatory` annotation — they require handler-level logic in the `submit`, `checkIn`, `rejectPass`, and `checkOut` action handlers.

PART 4 — BUSINESS RULES (BR-01 to BR-14)

BR-01: Visit date must not be in the past.

LAYMAN **Rule:** You cannot raise a pass for yesterday. If the visit date is before today, the system refuses.

TECHNICAL **When checked:** At **Save** (if validated) and at **Submit**. The submit handler compares `pass.visitDate < localDate()`. If true: error `BR01_PAST_DATE` — *"A pass cannot be raised for a date that has already passed."*

BR-02: The visit window must be valid.

LAYMAN **Rule:** Four checks in one:

1. `validTo` must be after `validFrom` (can't end before you start)
2. Both are on the same day (no overnight visits)
3. Maximum 12-hour window
4. Must fall inside the plant's gate hours (Chennai opens 07:00, closes 20:00)

TECHNICAL **When checked:** Save and Submit. The handler converts times to minutes-since-midnight and compares against `plant.gateOpenTime` / `plant.gateCloseTime`. Error messages: `BR02_WINDOW_ORDER`, `BR02_MAX_12H`, `BR02_GATE_HOURS`.

BR-03: Visitor details and active checks.

LAYMAN **Rule:** `visitorName` ≥ 3 characters. `visitorMobile` exactly 10 digits starting with 6–9. The host, plant, and area must all be marked as active.

TECHNICAL **When checked:** At Submit. The mobile regex is `/^[6-9]\d{9}$/`. Active checks query the master records: `if (host && !host.isActive)` → error `BR03_HOST_INACTIVE`. Employee E1006 (Deepak Nair) exists in the test data specifically to make this rule fire.

BR-04: Automatic routing — approval or straight to approved.

LAYMAN **Rule:** If the area has `requiresApproval = true`, submit sends the pass to `REQUESTED` (waiting for the supervisor). If `false`, it goes straight to `APPROVED`.

TECHNICAL **Why no error message?** Because this is a **determination, not a validation**. It doesn't refuse anything — it decides the outcome. One config column, two outcomes, no code change. The resulting status is obvious on the screen via the coloured badge.

```
const target = area?.requiresApproval ? 'REQUESTED' : 'APPROVED';
```

BR-05: Safety induction requirement.

LAYMAN **Rule:** If the area has `requiresInduction = true`, the pass must include an induction reference number (`inductionRef`) and a valid-until date (`inductionValidTill`) that is on or after the visit date.

TECHNICAL **Why checked twice?**

1. **At Submit** — validates against `visitDate`. Catches: "your induction expired last month."
2. **At Check-In** — validates against **today's date**. Catches: "your induction was valid when the pass was submitted last week, but it expired yesterday." The spec says: *"An expired induction cannot reach the shop floor."* The gate is the last line of defence.

BR-06: Check-in window.

LAYMAN **Rule:** A guard can check in a visitor starting 30 minutes before `validFrom` until `validTo`, on the visit date only. 31 minutes early = hard error. 29 minutes early = fine.

TECHNICAL **When checked:** At Check-In only. Error: `BR06_OUTSIDE_WINDOW` — *"This pass is valid from {0} to {1} on {2}. Ask the host to extend it."*

BR-07: Same visitor cannot be on-site twice.

LAYMAN **Rule:** The same mobile number cannot be `ON_SITE` on two passes at once, across all plants.

TECHNICAL **Why is it a warning at submit but an error at check-in?**

At **submit**, it's a **warning**: the host may legitimately be registering tomorrow's visit while the visitor is still inside today. Blocking that would be wrong. The host sees: *"Warning: this visitor already has an open pass today at {0}."*

At **check-in**, it's a **hard error**: if the guard admitted the same person on two passes, the muster list would show them twice — or, worse, checking out on one pass would still show them as "inside" on the other. The guard sees: *"Already on site at {0} on pass {1} — check them out there first."*

Same rule, same data, two severities at two moments. This is the most important rule to study.

BR-08: Badge uniqueness.

LAYMAN **Rule:** At check-in, a badge number is mandatory. That badge number must not be currently held by another `ON_SITE` pass. At check-out, the badge is released (set to `null`).

TECHNICAL **When checked:** At Check-In. Error: `BR08_BADGE_IN_USE` — *"Badge {0} is already issued to {1} on pass {2}."*

BR-09: Item verification at check-out.

LAYMAN **Rule:** Check-out is blocked while any returnable item is unconfirmed. The guard must either tick the item as returned (`returnedAtExit = true`) or provide a remark explaining why it's not leaving (`exitRemark`). This is how the "laptop walked out" disputes from P6 become resolvable.

TECHNICAL **When checked:** At Check-Out. Error: `BR09_ITEMS_UNCONFIRMED` — *"{0} returnable item(s) not confirmed: {1}. Confirm them, or record why they are not leaving."*

BR-10: Rejection needs a proper reason.

LAYMAN **Rule:** When a supervisor rejects a pass, they must give a reason of at least 10 characters. "No" is not enough — *"the host has to be able to act on it."*

TECHNICAL **When checked:** At Reject. Error: `BR10_REASON_SHORT`. The reason is stored on both the pass (`rejectionReason`) and the event row, and is read-only forever after.

BR-11: Nobody may approve their own pass.

LAYMAN **Rule:** Even if Meera holds both the Host and SecuritySupervisor roles, she cannot approve a pass where she is the host. This is **segregation of duties**.

TECHNICAL **Why it exists:** This is the first thing an auditor tests. If the same person can both request and approve, the entire approval workflow is meaningless. The test user `meera` exists specifically to test this. Error: `BR11_SELF_APPROVE` — *"You cannot approve a visitor you are hosting yourself."*

```
if (await isOwnPass(pass, req)) return fail(req, 403, 'BR11_SELF_APPROVE');
```

BR-12: Pass number generation and event logging — two parts.

LAYMAN **Rule — Part 1:** `passNo` is generated at submit in the format `GP---<5 digits>`, per plant per year, never reused.

LAYMAN **Rule — Part 2:** Every status change writes exactly one `PassEvent` row **in the same transaction**. If the event row cannot be written, the status change rolls back too — status and audit commit together or not at all.

TECHNICAL **Technical:** The `generatePassNo()` function queries the last passNo with that prefix, increments it, and pads to 5 digits. The `writeEvent()` function runs on the same request transaction (CAP's default — no explicit `tx.begin()` / `tx.commit()` needed because the entire `on` handler runs in one transaction).

BR-13: Copy to a new draft.

LAYMAN **Rule:** Any pass (in any status) can be copied into a new DRAFT. Visitor details, ID proof, and items carry over. Pass number, status, dates, badge, and timestamps do **not** copy. The host sees: *"Pass copied from {0}. Check the dates and the induction validity before submitting."*

TECHNICAL **Technical:** The `copy` action uses `this.send({ event: 'NEW' })` to create a draft (not an active record), then inserts the items into the draft's items table. This ensures the copy behaves exactly like a new draft — private, invisible to other users, no pass number.

BR-14: ID proof number masking.

LAYMAN **Rule:** The `idProofNumber` (e.g., Aadhaar number) is returned **masked** (showing only the last 4 characters) to everyone except:

- The **guard** — but only when the pass is in `APPROVED` status (at the moment of check-in, when they need to compare the number against the physical ID card)
- The **Admin** — who has full access for administrative purposes

TECHNICAL **Where implemented:** In an `after('READ')` handler on the **server**, never in the UI:

```
const maskIdProof = (row, req) => {
  if (!row?.idProofNumber) return;
  if (req.user.is('Admin')) return;
  if (req.user.is('SecurityGuard') && row.status_code === 'APPROVED') return;
  row.idProofNumber = '****' + row.idProofNumber.slice(-4);
};
```

PART 5 — STATUS LIFECYCLE & AUTHORIZATION

5.1 The eight statuses and the normal path.

```
DRAFT → REQUESTED → APPROVED → ON_SITE → CHECKED_OUT
```

Status	Meaning	Criticality (colour)
`DRAFT`	Host is still typing. Private, no rules applied. No passNo.	—
`REQUESTED`	Submitted, waiting for supervisor approval.	2 (amber)
`APPROVED`	Ready for the visitor to arrive at the gate.	3 (green)
`REJECTED`	Supervisor refused. Terminal — never reopened.	1 (red)
`ON_SITE`	Visitor is physically inside the plant.	5 (blue)
`CHECKED_OUT`	Visitor has left. Terminal.	3 (green)
`CANCELLED`	Host called off the visit. Terminal.	—
`EXPIRED`	Approved pass whose validity passed and visitor never showed up. Terminal.	1 (red)

Shortcut path: If `area.requiresApproval = false`, submit goes straight from DRAFT → APPROVED, skipping REQUESTED.

5.2 Which transition is NOT allowed, and why?

ON_SITE → CANCELLED does not exist and never will.

LAYMAN **Layman:** A person is physically inside a factory. You can't "cancel" their presence — they're standing there. The only way a pass in ON_SITE ends is a human at the gate pressing Check Out. Inventing a cancellation for someone who is inside would be inventing a departure that never happened — and that's exactly the P3 problem the entire app exists to solve.

TECHNICAL **Technical:** The transition is simply absent from `ACTION_MAP`. Any attempt to call `cancel` on an ON_SITE pass will hit `assertTransition()` and get `409 INVALID_STATUS`.

5.3 Why is a cancelled pass kept rather than deleted?

LAYMAN **Layman:** Deleting it destroys the audit trail. If Anitha raised 5 passes this month and cancelled 2, the report should show "5 raised, 2 cancelled" — not "3 raised." The cancelled pass stays visible and countable. It just can't be used anymore.

TECHNICAL **Technical:** Cancellation sets `status_code = 'CANCELLED'` and writes a `PassEvent`. The pass remains in the database with all its data intact. This is also why `CANCELLED` is a terminal status — no further transitions are allowed.

5.4 What does the day-end job do, and why doesn't it close a visitor who is still on site?

LAYMAN **Rule:** The day-end job (`runDayEndJob`) does two things:

1. **Expire** — any `APPROVED` pass whose validity has passed and was never used → `EXPIRED`
2. **Flag overstay** — any `ON_SITE` pass whose `validTo` has passed → set `isOverstay = true`, but status **stays ON_SITE**

LAYMAN **Why not auto-close?** Because the visitor is still inside the building. If the system set `actualOutAt = 21:00` and changed the status to `CHECKED_OUT`, the muster list would say "nobody inside" when someone actually is. That's P3 with a computer behind it. The overstay flag makes them appear on the supervisor's overstay report (R2), prompting a human to go find the visitor and walk them to the gate.

TECHNICAL **Technical:** The job is idempotent — running it twice changes nothing the second time (the filter includes `isOverstay: false` for the overstay query). The Admin runs it once each evening. Returns `{ expired: N, overstay: M }`.

5.5 Role-based vs. instance-based authorization in CAP.

LAYMAN **Layman:**

- **Role-based** = "Are you a guard? Then you can check people in." It's like a job title.
- **Instance-based** = "Are you the guard at *this specific plant*?" or "Is this *your own* pass?" It's like checking your badge number against the door number.

TECHNICAL **Technical:**

- **Role-based:** `@requires: 'Admin'` or `grant: 'READ', to: 'Host'` — checks if `req.user.is('Admin')`.
- **Instance-based:** `where: 'createdBy = $user'` — the CAP runtime evaluates this against the logged-in user and adds it to the SQL WHERE clause. The Host can only see passes where `createdBy` matches their user ID. The Guard can only see passes where `plant.code` matches their `$user.plant` attribute.

The `@restrict` annotation in gatepass-service.cds combines both.

5.6 A second host opens another host's pass by typing the URL directly. What must come back?

LAYMAN **Layman:** They must get **403 Forbidden** or an **empty result** — never the actual data. Hiding the button in the UI is not enough. The spec calls this out explicitly: *"'the button is hidden' is not an answer."* A URL can be typed, a curl command can be crafted, a test tool can be pointed. The rule must live on the **server**, not on the **screen**.

TECHNICAL **Technical:** The `where: 'createdBy = $user'` clause on the Host's `READ` grant means the CAP runtime adds a SQL filter. When `karthik` requests `anitha`'s pass by ID, the generated query includes `WHERE createdBy = 'karthik'` — which matches zero rows, so the response is empty (or 404). TC-09 tests this explicitly.

PART 6 — CAP IMPLEMENTATION PATTERNS

6.1 CAP has before, on, and after handlers. What would you use each one for?

Phase	What it does	Examples in this app
`before`	Runs **before** the default/custom operation. Used for **validation** and **input manipulation**. If it calls `req.reject()`, the operation never happens.	Strip system fields from PATCH (`stripSystemFields`), set host and status on NEW draft, prevent deletion of master data, enforce DRAFT-only editing
`on`	**Replaces** the default operation. Used for **custom actions** and **complex business logic**.	All seven actions: `submit`, `approve`, `rejectPass`, `cancel`, `checkIn`, `checkOut`, `copy`, plus `runDayEndJob` and `confirmReturn`
`after`	Runs **after** the operation succeeded. Used for **enrichment** and **post-processing**. Cannot prevent the operation.	BR-14 masking of `idProofNumber`, computing `ageingDays` virtual field

---

6.2 How do you define, implement, and call an action such as submit?

Step 1 — Define in CDS (gatepass-service.cds:109):

```
entity VisitorPasses as projection on ... {
  ...
} actions {
  action submit() returns VisitorPasses;
};
```

Step 2 — Implement in JS (gatepass-service.js:330):

```
this.on('submit', VisitorPasses, async (req) => {
  const pass = await loadPass(req);
  assertTransition(pass.status_code, 'submit', req);
  // ... validation and business logic ...
  await UPDATE(DBPasses(), ID).set({ status_code: target, passNo });
  await writeEvent(ID, 'DRAFT', target, 'SUBMIT', req);
  return respond(ID, req);
});
```

Step 3 — Call from UI or API:

```
POST /odata/v4/gate-pass/VisitorPasses(ID=<uuid>)/GatePassService.submit
```

Fiori Elements renders this as a button via `@UI.LineItem` or `@UI.Identification` annotations.

6.3 What does draft-enabling VisitorPasses give you, and what must be true of a draft?

LAYMAN **Layman:**

Draft-enabling means the host can start filling in a pass, close the browser, come back the next day, and continue where they left off. The half-finished pass is saved automatically but **nobody else can see it** — not other hosts, not the admin, not anyone. It only becomes visible when the host saves (activates) it.

TECHNICAL **Technical:**

`@odata.draft.enabled` on the VisitorPasses projection:

1. Creates a shadow `_drafts` table in the database
2. Enables auto-save on every field change in Fiori Elements
3. Provides `NEW`, `EDIT`, `SAVE` (activation), `CANCEL` draft events
4. Drafts are isolated by `DraftAdministrativeData.InProcessByUser` — the M2 gate says: *"A draft is invisible to every other user including Admin — prove it in a second browser."*

What must be true: a draft has no `passNo` (assigned at submit), no business rules applied yet, status is `DRAFT`, and it's private to the creator.

6.4 BR-07 and BR-08 cannot be written as simple field checks. How are they implemented?

LAYMAN **Layman:**

These rules require **looking at other records**, not just checking the current pass's own fields. BR-07 asks: "Is this same mobile number already on-site on another pass?" BR-08 asks: "Is this badge number already held by someone else who's on-site?" You can't answer either question by looking at the pass alone — you have to query the whole database.

TECHNICAL **Technical:**

BR-07 — Cross-pass duplicate check (gatepass-service.js:543-545):

```
const onSite = await SELECT.one.from(DBPasses())
  .where({ visitorMobile: pass.visitorMobile, status_code: 'ON_SITE', ID: { '≠': ID } });
if (onSite) err('BR07_ON_SITE_ERROR', [onSite.plant_code, onSite.passNo]);
```

BR-08 — Cross-pass badge check (gatepass-service.js:549-551):

```
const held = await SELECT.one.from(DBPasses())
  .where({ badgeNo, status_code: 'ON_SITE', ID: { '≠': ID } });
if (held) err('BR08_BADGE_IN_USE', [badgeNo, held.visitorName, held.passNo]);
```

Both use `SELECT.one` queries across all passes (not just the current one), filtering by `status_code = 'ON_SITE'` and excluding the current pass's ID.

6.5 Why must the status change and the PassEvent row be written in the same transaction, and how?

LAYMAN **Layman:**

Imagine you update the pass to "ON_SITE" but the event log entry fails. Now the pass says the visitor is inside, but there's no audit record of who admitted them or when. Or the reverse: the event says they were admitted but the pass still says "APPROVED." Either way, the data is inconsistent and untrustworthy. Both must succeed together or fail together.

TECHNICAL **Technical:**

In CAP, every `on` handler for a bound action runs within the **request's implicit transaction**. Both `UPDATE(DBPasses())` and `INSERT.into(DBEvents())` use the same underlying database transaction. If anything throws after the UPDATE but before the INSERT completes, the entire transaction rolls back. No explicit `tx.begin()` / `tx.commit()` is needed — CAP does this automatically for each request.

The spec says: *"Kill the process between the status update and the event insert: the status change rolled back too."*

6.6 Where do the CSV sample files go, how are they named, and what must be true after cds deploy?

LAYMAN **Layman:**

CSV files (think: spreadsheets) containing test data go in the `db/data/` folder. Each file is named to match the database table it fills: `nub.gatepass-Plants.csv` fills the Plants table, `nub.gatepass-Employees.csv` fills the Employees table, etc. After running `cds deploy`, all the data from these CSVs must be loaded with no skipped rows.

TECHNICAL **Technical:**

CAP convention: files in `db/data/` named `-.csv` are automatically loaded by `cds deploy`. The namespace is `nub.gatepass`, contexts add dot-separated segments:

- `nub.gatepass.master-Plants.csv`
- `nub.gatepass.master-Employees.csv`
- `nub.gatepass.master-VisitAreas.csv`
- `nub.gatepass.config-VisitPurposes.csv`
- `nub.gatepass.config-IdProofTypes.csv`
- `nub.gatepass.config-PassStatus.csv`
- `nub.gatepass.transaction-VisitorPasses.csv`
- etc.

After deploy: every CSV row exists in the database, foreign keys resolve, and the test scenarios from Section 12 are executable.

6.7 How do you test this application, and how is it deployed to BTP?

Testing:

LAYMAN **Layman:**

Automated tests run with `npm test`. Each test logs in as a specific user (anitha, karthik, guard1, etc.) and simulates creating passes, submitting, checking in, etc. The tests prove every business rule works — not by clicking buttons, but by sending the same requests a browser would send.

TECHNICAL **Technical:**

Tests use `@cap-js/cds-test` with Node's built-in test runner (`node --test test/*.test.js`). Each test case corresponds to a TC-id from Section 13. The test authenticates as a specific user, sends OData requests, and asserts the response status codes and data. Key tests:

- TC-08: raw PATCH to system fields → refused
- TC-09: cross-user access → 403 or empty
- TC-10: update/delete PassEvent → refused

Deployment:

LAYMAN **Layman:**

The application is packaged into an "MTA" (Multi-Target Application) archive and deployed to SAP BTP. On BTP, it uses HANA Cloud for the database and XSUAA for real user authentication instead of the mocked users used locally.

TECHNICAL **Technical:**

The `mta.yaml` defines modules (srv, db deployer, UI apps) and resources (HANA HDI container, XSUAA service instance, destinations). Build: `mbt build` → produces `.mtar` in `mta_archives/`. Deploy: `cf deploy .mtar`. The `[production]` profile in `package.json` switches auth from `mocked` to `xsuaa`. The `xs-security.json` defines the role templates (`Host`, `SecurityGuard`, `SecuritySupervisor`, `Admin`) and role collections.

SUMMARY TABLE: Key Design Decisions at a Glance

Decision	Why
BTP side-app, not standard SAP	No SAP transaction covers visitor gate management
`requiresApproval` is configuration, not code	EHS changes shouldn't require a developer
passNo at submit, not at draft	Drafts are tentative; numbers are official
PassEvents as a separate immutable table	Full audit trail, not just "last modified by"
One transition table, not scattered checks	Single source of truth, one place to audit
ON_SITE → CANCELLED forbidden	A person inside a factory can't be "cancelled"
Day-end flags overstay, never auto-closes	The system must never invent a departure time (P3)
Rules on the server, not the UI	`curl` and direct URLs bypass UI controls
Master data soft-deleted with `isActive`	Old passes must still resolve their references
ID proof masked on the server	BR-14 cannot be defeated by reading raw OData

GatePass — Comprehensive Q&A Document
Generated from the GatePass project specification and codebase.
NubTech Industries Pvt. Ltd. — Plant Security & EHS · SAP BTP CAP Application